FATEC BS Rubens Lara

Curso de Ciência de Dados

Disciplina: Masoquismo Funcional

Soma Simbólica Formal em Haskell

Uma Formalização de Máquina de Mealy
para Adição Decimal com State Monad
e Categoria de Kleisli

Autor

Peterson Carara Junior

Professor responsável

João Paulo

Santos – SP

23 de setembro de 2026

Resumo

Este documento descreve a formalização, em Haskell, de uma Máquina de Estados Finitos (FSM) de Mealy para a soma simbólica de números decimais. O projeto evolui em três camadas: (i) definição dos Tipos Algébricos de Dados (ADTs) que modelam dígitos e o estado de transporte (*vai-um*); (ii) encapsulamento do estado do autômato em uma *State Monad* customizada; e (iii) composição de transições na *Categoria de Kleisli*. Um princípio norteador do trabalho é que a operação de soma permaneça *inteiramente no domínio dos símbolos finitos*, sem conversões para tipos numéricos nativos. O documento inclui o diagrama formal da FSM, o código comentado, a fundamentação categorial e um glossário detalhado de todos os termos técnicos utilizados.

Palavras-chave: Programação Funcional. Haskell. Máquina de Mealy. State Monad. Categoria de Kleisli. Teoria das Categorias. Pureza Simbólica.

Sumário

1 Introdução **3**
1.1 Princípio da Pureza Simbólica 3

2 Fundamentação Teórica **3**
2.1 Definição Formal da FSM de Mealy 3
2.2 Tipos Soma como Coprodutos 4
2.3 State Monad e Categoria de Kleisli 4

3 Diagrama da FSM **4**
3.1 Leitura do Diagrama 4

4 Código Comentado **5**
4.1 Tipos Algébricos de Dados (ADTs) 5
4.2 Conversões de Fronteira (I/O) 5
4.3 Função de Transição de Mealy 5
4.4 State Monad Customizada 6
4.5 Transição Monádica 6
4.6 Composição de Kleisli 6
4.7 Alinhamento e Processamento 7

5 Propriedades e Análise **8**
5.1 Completude Funcional 8
5.2 Segurança de Tipos 8
5.3 Totalidade Funcional 8
5.4 Associatividade da Composição Monádica 8
5.5 Ausência de Conversões Numéricas 8

6 Glossário Detalhado de Termos **8**
6.1 Termos da Teoria das Categorias 8
6.2 Termos de Haskell 10
6.3 Termos Específicos do Projeto 12

7 Conclusão **13**

1 Introdução

O objetivo deste projeto é formalizar, em Haskell, o algoritmo clássico de soma decimal posição-a-posição, tratando-o como uma **Máquina de Mealy** cujo estado é o transporte (*vai-um*) e cuja entrada é o par de dígitos na mesma posição.

A escolha de Haskell não é acidental: a linguagem oferece três propriedades que sustentam a formalização:

1. **Tipos algébricos** que modelam diretamente os conjuntos finitos Q e Σ da definição formal de FSM.
2. **Pattern matching exaustivo**, que garante a completude funcional em tempo de compilação.
3. **Type safety**, segundo a qual "erros de tipo nunca podem ocorrer durante a avaliação" [1].

1.1 Princípio da Pureza Simbólica

Um dos princípios norteadores deste projeto é que a operação de soma permaneça **inteiramente no domínio dos símbolos finitos**. Isso significa que, em nenhum momento da execução, os dígitos são convertidos para tipos numéricos como `Int` ou `Integer`.

Formalmente, seja $\mathcal{A} = \{\texttt{D0}, \texttt{D1}, \ldots, \texttt{D9}\}$ o alfabeto interno de dígitos. A função de transição $\delta : \mathcal{A} \times \mathcal{A} \times Q \to \mathcal{A} \times Q$ é definida como uma **tabela explícita de regras simbólicas**, e não como uma fórmula aritmética. Cada linha da definição de `somar` corresponde a um par ordenado $(d_1, d_2) \mapsto (r, v)$, onde r é o dígito resultante e v o novo estado de transporte.

A função `charParaDigito` *não* é uma conversão para número: é uma projeção do alfabeto Unicode ($|\texttt{Char}| \approx 1.1 \times 10^6$) para o alfabeto finito $\mathcal{A}$ ($|\mathcal{A}| = 10$). Ambos os alfabetos são igualmente simbólicos; a projeção apenas restringe o conjunto de símbolos admissíveis.

Essa escolha tem três consequências importantes:

1. **Pureza referencial**: não há dependência de tipos numéricos nativos nem de operações aritméticas da máquina.
2. **Transparência estrutural**: a função de transição é literalmente uma tabela finita, diretamente verificável por inspeção e compatível com o modelo formal de Máquina de Mealy.
3. **Totalidade funcional**: a conversão de `Char` para `Digito` é total, retornando `Nothing` para caracteres inválidos, evitando exceções em tempo de execução.

2 Fundamentação Teórica

2.1 Definição Formal da FSM de Mealy

Uma Máquina de Mealy é uma 6-tupla $\mathcal{M} = (Q, \Sigma, \Lambda, \delta, \lambda, q_0)$, onde Q é o conjunto finito de estados, Σ o alfabeto de entrada, Λ o alfabeto de saída, $\delta : Q \times \Sigma \to Q$ a função de transição, $\lambda : Q \times \Sigma \to \Lambda$ a função de saída e $q_0 \in Q$ o estado inicial.
Para a soma decimal:

$$
\begin{aligned}
Q &= \{\texttt{SemVaiUm}, \texttt{ComVaiUm}\},\\
\Sigma &= \texttt{Digito} \times \texttt{Digito},\\
\Lambda &= \texttt{Digito},\\
q_0 &= \texttt{SemVaiUm}.
\end{aligned}
$$

2.2 Tipos Soma como Coprodutos

Na categoria **Hask**, os tipos de Haskell são objetos e as funções são morfismos. Um tipo soma como `data Digito = D0 | D1 | ... | D9` é o **coproduto** de dez cópias do objeto terminal, isto é, $10 = 1 + 1 + \cdots + 1$ [2]. Isso garante que a finitude exigida por Q e Σ seja preservada com fidelidade.

2.3 State Monad e Categoria de Kleisli

A **State Monad** encapsula uma função pura de transformação de estado [1]:

$$\texttt{ST}\ s\ a\ \cong\ s \to (a, s). \tag{1}$$

Os morfismos da **Categoria de Kleisli** associada são funções $a \to \texttt{ST}\ s\ b$, compostas pelo operador *fish* $>=>$ [2]. No código, isso é implementado por `kleisliComp`.

3 Diagrama da FSM

A Figura 1 apresenta o diagrama de estados da Máquina de Mealy para a soma decimal. Cada transição é rotulada no formato *entrada [condição] / saída*.

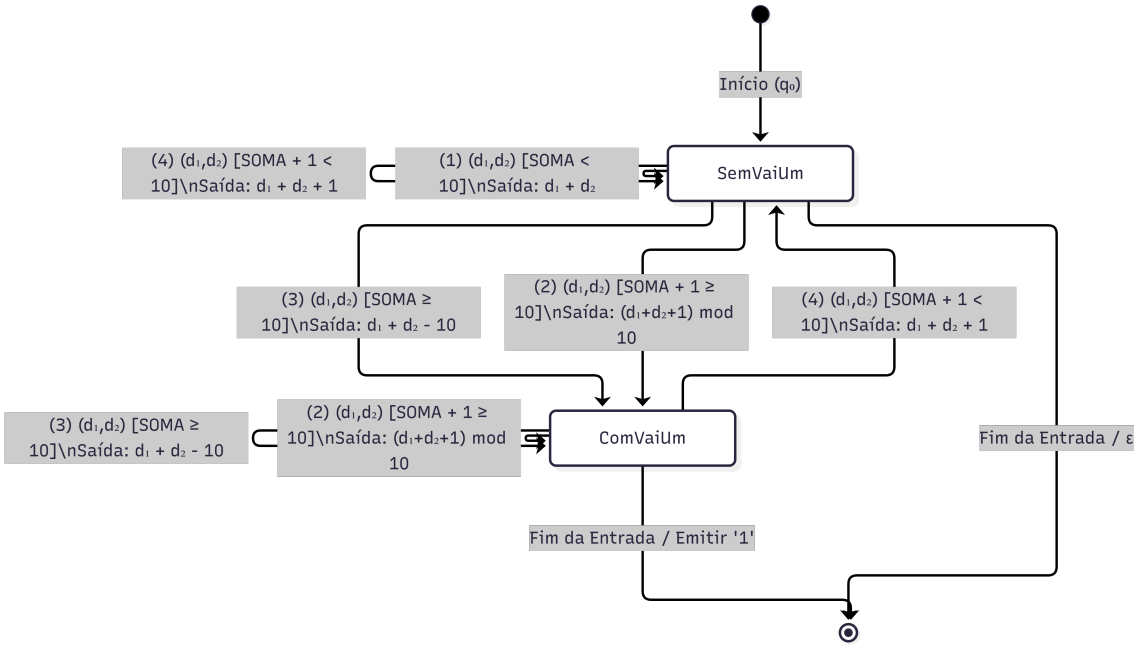


Figura 1: Diagrama de estados da Máquina de Mealy para soma decimal. O estado inicial é `SemVaiUm` (q_0); o estado final é representado pelo círculo duplo. As transições (1)–(4) descrevem o comportamento completo do autômato.

3.1 Leitura do Diagrama

As quatro transições rotuladas de (1) a (4) descrevem o comportamento completo do autômato:

- **(1)** Soma simples sem transporte, resultado < 10: permanece em `SemVaiUm`.
- **(2)** Soma com transporte de entrada, resultado $+1 \geq 10$: permanece em `ComVaiUm`, emitindo o dígito truncado $(d_1 + d_2 + 1) \bmod 10$.
- **(3)** Soma simples ≥ 10: transita para `ComVaiUm`, emitindo $d_1 + d_2 - 10$.

- **(4)** Soma com transporte de entrada, resultado $+1 < 10$: retorna a `SemVaiUm`, emitindo $d_1 + d_2 + 1$.

Ao final da entrada, se o estado for `ComVaiUm`, emite-se o dígito `'1'` adicional; caso contrário, emite-se a palavra vazia ε.

4 Código Comentado

4.1 Tipos Algébricos de Dados (ADTs)

```haskell
data Digito = D0 | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 | D9
  deriving (Show, Eq)

data VaiUm = SemVaiUm | ComVaiUm
  deriving (Show, Eq)
```

Listing 1: Definição dos ADTs `Digito` e `VaiUm`.

Esses dois tipos modelam, respectivamente, o alfabeto de símbolos (dígitos) e o conjunto de estados da FSM (transporte). Ambos são **coprodutos finitos** na categoria **Hask** [2].

4.2 Conversões de Fronteira (I/O)

```haskell
digitoParaChar :: Digito -> Char
digitoParaChar D0 = '0'
digitoParaChar D1 = '1'
-- ... (demais casos) ...
digitoParaChar D9 = '9'

charParaDigito :: Char -> Maybe Digito
charParaDigito '0' = Just D0
charParaDigito '1' = Just D1
-- ... (demais casos) ...
charParaDigito '9' = Just D9
charParaDigito _   = Nothing
```

Listing 2: Conversões totais entre `Digito` e `Char`.

A conversão `charParaDigito` é **total**, retornando `Nothing` para caracteres inválidos, em vez de lançar uma exceção. Isso mantém o princípio de pureza simbólica: nenhuma exceção é lançada em tempo de execução, e o tratamento de erros é feito via o tipo `Maybe`, que é ele mesmo um tipo algébrico.

4.3 Função de Transição de Mealy

```haskell
transicaoMealy :: (Digito, Digito) -> VaiUm -> (Digito, VaiUm)
transicaoMealy (d1, d2) v = somarComCarry d1 d2 v
  where
    somar :: Digito -> Digito -> (Digito, VaiUm)
    somar D0 x  = (x, SemVaiUm)
    somar x D0  = (x, SemVaiUm)
    somar D1 D1 = (D2, SemVaiUm)
    -- ... (demais casos) ...
    somar D9 D9 = (D8, ComVaiUm)
    somar x y   = somar y x   -- comutatividade

```

```
    somarComCarry :: Digito -> Digito -> VaiUm -> (Digito, VaiUm)
    somarComCarry x y SemVaiUm = somar x y
    somarComCarry x y ComVaiUm =
      let (r1, v1) = somar x y
          (r2, v2) = somar r1 D1
      in (r2, combinarVaiUm v1 v2)

    combinarVaiUm :: VaiUm -> VaiUm -> VaiUm
    combinarVaiUm ComVaiUm _ = ComVaiUm
    combinarVaiUm _ ComVaiUm = ComVaiUm
    combinarVaiUm _ _        = SemVaiUm
```

Listing 3: Função de transição `transicaoMealy`.

O **pattern matching exaustivo** sobre todos os pares de `Digito` garante a **completude funcional** em tempo de compilação [1]. Note que `somar` *não* executa aritmética: cada linha é uma regra simbólica da tabela de transição.

4.4 State Monad Customizada

```
newtype ST s a = ST { runST :: s -> (a, s) }

instance Functor (ST s) where
  fmap g st = ST $ \s ->
    let (x, s') = runST st s
    in (g x, s')

instance Applicative (ST s) where
  pure x = ST $ \s -> (x, s)
  stf <*> stx = ST $ \s ->
    let (f, s')  = runST stf s
        (x, s'') = runST stx s'
    in (f x, s'')

instance Monad (ST s) where
  st >>= f = ST $ \s ->
    let (x, s') = runST st s
    in runST (f x) s'
```

Listing 4: State Monad `ST` e suas instâncias.

Essa estrutura é diretamente baseada em [1], onde o "estado" da mônada é instanciado pelo transporte do autômato.

4.5 Transição Monádica

```
pasoMealyMonad :: (Digito, Digito) -> ST VaiUm Digito
pasoMealyMonad par = ST $ \estadoAtual ->
  let (digitoSaida, novoEstado) = transicaoMealy par estadoAtual
  in (digitoSaida, novoEstado)
```

Listing 5: Transição monádica `pasoMealyMonad`.

4.6 Composição de Kleisli

```haskell
kleisliComp :: (a -> ST s b) -> (b -> ST s c) -> (a -> ST s c)
kleisliComp f g = \x -> f x >>= g
```

Listing 6: Operador de composição de Kleisli.

Esse operador é o *fish* da Categoria de Kleisli associada à State Monad [2].

4.7 Alinhamento e Processamento

```haskell
alinharDigitos :: [Digito] -> [Digito] -> ([(Digito, Digito)], [Digito])
alinharDigitos xs ys = (zip xsPadded ysPadded, [])
  where
    maxLen = max (length xs) (length ys)
    pad l  = l ++ replicate (maxLen - length l) D0
    xsPadded = pad xs
    ysPadded = pad ys

processarSoma :: [(Digito, Digito)] -> ST VaiUm [Digito]
processarSoma [] = do
  v <- ST $ \s -> (s, s)
  case v of
    ComVaiUm -> return [D1]
    SemVaiUm -> return []
processarSoma (p:ps) = do
  d  <- pasoMealyMonad p
  ds <- processarSoma ps
  return (d : ds)

-- | Converte uma String em uma lista de Digitos, se possivel.
stringParaDigitos :: String -> Maybe [Digito]
stringParaDigitos = traverse charParaDigito

-- | Soma duas Strings usando o automato monadico.
-- Retorna Nothing se alguma entrada contiver caractere invalido.
somaDois :: String -> String -> Maybe String
somaDois s1 s2 = do
  d1 <- stringParaDigitos (reverse s1)
  d2 <- stringParaDigitos (reverse s2)
  let (pares, _) = alinharDigitos d1 d2
      (resDigitos, _) = runST (processarSoma pares) SemVaiUm
      resString = map digitoParaChar (reverse resDigitos)
  return (if null resString then "0" else resString)

-- | Soma uma lista de Strings via reducao com foldl1.
somaLista :: [String] -> Maybe String
somaLista [] = Just "0"
somaLista (x:xs) = foldl step (Just x) xs
  where
    step acc y = acc >>= \a -> somaDois a y
```

Listing 7: Alinhamento de dígitos e soma completa.

A função `processarSoma` utiliza a **notação do** (que é açúcar sintático para `>>=`) para sequenciar as transições monádicas. A função `stringParaDigitos` usa `traverse` para converter uma `String` em `Maybe [Digito]`, propagando automaticamente a falha caso algum caractere seja inválido. A função `somaLista` generaliza a operação para N números via `foldl` monádico.

5 Propriedades e Análise

5.1 Completude Funcional

A exaustividade do pattern matching em `somar` garante que **nenhum par de dígitos** deixa de ser tratado. O compilador GHC emite aviso (ou erro, com `-Werror`) caso algum caso seja omitido [1].

5.2 Segurança de Tipos

Segundo Hutton [1], "a inferência de tipos precede a avaliação", de modo que "erros de tipo nunca podem ocorrer durante a avaliação". Neste projeto, isso se manifesta em três níveis:

1. Impossibilidade de representar um dígito inválido.
2. Impossibilidade de aplicar `transicaoMealy` a algo que não seja um par de `Digito`.
3. Impossibilidade de compor funções incompatíveis sem que o compilador detecte.

5.3 Totalidade Funcional

Todas as funções do projeto são **total**: retornam um valor bem definido para toda entrada válida e sinalizam entradas inválidas via `Maybe`, em vez de lançar exceções. Isso é uma consequência direta do princípio de pureza simbólica: nenhuma operação parcial é executada, nem em `charParaDigito`, nem em `somaDois`, nem em `somaLista`.

5.4 Associatividade da Composição Monádica

Como a State Monad satisfaz as leis das mônadas [1], a composição via `kleisliComp` é associativa:

$$(f \gg g) \gg h \;=\; f \gg (g \gg h). \tag{2}$$

Isso garante que a ordem de agrupamento das transições não afeta o resultado final, propriedade essencial para a soma de listas de números.

5.5 Ausência de Conversões Numéricas

Um aspecto crucial do projeto é que **em nenhum ponto** os dígitos são convertidos para `Int`, `Integer`, ou qualquer tipo numérico. A única conversão realizada é entre dois alfabetos simbólicos distintos (`Char` e `Digito`), ambos finitos e discretos. Isso preserva o caráter estritamente simbólico da operação.

6 Glossário Detalhado de Termos

Esta seção apresenta uma explicação detalhada de todos os termos técnicos, categóricos e de Haskell utilizados ao longo do documento.

6.1 Termos da Teoria das Categorias

Categoria (Hask)
Uma categoria $\mathcal{C}$ consiste em uma coleção de *objetos* e uma coleção de *morfismos* entre eles, com uma operação de composição associativa e uma identidade para cada objeto. A categoria **Hask** tem como objetos os tipos de Haskell e como morfismos as funções entre esses tipos [2].

Objeto
Elemento de uma categoria. Em **Hask**, um objeto é um tipo. Por exemplo, `Digito` é um objeto.

Morfismo
Uma "seta" $f : A \to B$ entre dois objetos. Em **Hask**, um morfismo é uma função.

Composição ($\circ$)
Operação que combina dois morfismos $f : A \to B$ e $g : B \to C$ em um terceiro $g \circ f : A \to C$. Em Haskell, é o operador `(.)`.

Identidade (`id`)
Para cada objeto A, existe um morfismo $\mathrm{id}_A : A \to A$ que é neutro para a composição. Em Haskell, `id :: a -> a`.

Coproduto ($+$)
Generalização categórica da *união disjunta*. Em **Hask**, o coproduto de dois tipos é um *tipo soma*.

Produto ($\times$)
Generalização categórica do *produto cartesiano*. Em **Hask**, o produto de dois tipos é uma *tupla*.

Objeto Terminal
Objeto para o qual existe um único morfismo vindo de qualquer outro objeto. Em **Hask**, é o tipo `()`.

Functor
Um mapeamento entre categorias que preserva a estrutura. Em Haskell, é a classe `Functor` com o método `fmap`.

Endofunctor
Functor de uma categoria para ela mesma.

Transformação Natural
Um mapeamento entre dois functores que respeita a estrutura categórica.

Mônada
Um endofunctor T equipado com duas transformações naturais $\eta : I \to T$ (unidade) e $\mu : T \circ T \to T$ (multiplicação), satisfazendo leis de associatividade e identidade [2, 1].

Categoria de Kleisli
Para uma mônada M, a categoria $\mathcal{C}_M$ tem os mesmos objetos de $\mathcal{C}$, mas os morfismos são funções $A \to MB$. A composição é dada por `>=>` (fish) e a identidade por `return` [2].

Operador *Fish* ($\gg$)
Operador de composição da Categoria de Kleisli. Em Haskell, `(>=>) :: (a -> m b) -> (b -> m c) -> (a -> m c)`.

Currying
Transformação que converte uma função de múltiplos argumentos em uma sequência de funções de um argumento: $(a, b) \to c \;\cong\; a \to (b \to c)$.

Isomorfismo ($\cong$)
Morfismo invertível.

Functor Representável
Functor naturalmente isomórfico a um hom-functor; funciona como um "contêiner de resultados memoizados" indexado por um tipo-chave.

6.2 Termos de Haskell

Tipo Algébrico de Dados (ADT)
Tipo definido pela composição de produtos e coprodutos. Usa-se `data` ou `newtype`.

Tipo Soma
Tipo cujos valores são um entre vários construtores, definido com `|`. Exemplo: `data Digito = D0 | D1 | ... | D9`.

Tipo Produto
Tipo cujos valores combinam múltiplos campos. Exemplo: `(Digito, VaiUm)`.

`newtype`
Declaração de um tipo com um único construtor e um único campo.

`data`
Declaração de um tipo algébrico com um ou mais construtores.

`deriving`
Gera automaticamente instâncias de classes como `Show` e `Eq`.

`Show`
Classe de tipos que podem ser convertidos para `String`.

`Eq`
Classe de tipos que suportam comparação de igualdade.

`Maybe`
Tipo soma que representa uma computação que pode falhar: `Just a` para sucesso, `Nothing` para falha.

`traverse`
Função que percorre uma estrutura aplicando uma função que produz um contexto aplicativo (como `Maybe`), coletando os resultados.

Pattern Matching
Mecanismo que compara um valor contra padrões sintáticos.

Pattern Matching Exaustivo
Quando todas as possibilidades de construtores são cobertas.

Função Pura
Função que sempre produz o mesmo resultado para o mesmo argumento e não tem efeitos colaterais.

Função Total
Função definida para todos os valores possíveis de seu domínio.

Type Safety
Erros de tipo nunca ocorrem em tempo de execução.

Type Inference
O compilador deduz os tipos das expressões automaticamente.

`::`
Anotação de tipo (“tem o tipo”).

`->`
Seta de função (associativa à direita).

`=` Definição.

`|` Separador de construtores em tipos soma.

`where`
Cláusula para definições locais.

`let ... in`
Definições locais dentro de uma expressão.

`do`
Notação para sequenciar ações monádicas.

`<-`
Gerador na notação `do`.

`return`
Injeção de valor puro em uma mônada.

`>>=`
Operador *bind* para encadear computações monádicas.

`>=>`
Operador *fish* (composição de Kleisli).

`fmap`
Mapeamento de functor.

`pure`
Equivalente a `return` no contexto `Applicative`.

`<*>`
Aplicação aplicativa.

`foldl`
Redução à esquerda sobre uma lista.

`foldl1`
Redução à esquerda sobre lista não-vazia.

`zip`
Combina duas listas em pares.

`reverse`
Inverte uma lista.

`replicate`
Produz lista com n cópias de um valor.

`map`
Aplica função a cada elemento de uma lista.

`instance`
Declaração de instância de classe.

6.3 Termos Específicos do Projeto

FSM (Finite State Machine)
Máquina de Estados Finitos.

Máquina de Mealy
Variante de FSM em que a saída depende do estado e da entrada.

Estado (Q)
Conjunto finito de configurações internas do autômato.

Alfabeto de Entrada (Σ)
Conjunto de símbolos que o autômato consome.

Alfabeto de Saída (Λ)
Conjunto de símbolos que o autômato emite.

Função de Transição (δ)
$\delta : Q \times \Sigma \to Q$.

Função de Saída (λ)
$\lambda : Q \times \Sigma \to \Lambda$.

Estado Inicial (q_0)
Estado em que o autômato começa.

Vai-Um (Carry)
Transporte que ocorre quando a soma de dois dígitos é ≥ 10.

State Monad
Mônada que encapsula transformações de estado de forma pura.

Pureza Simbólica
Princípio segundo o qual a operação permanece inteiramente no domínio dos símbolos finitos, sem conversões para tipos numéricos.

`pasoMealyMonad`
Versão monádica da função de transição.

`kleisliComp`
Implementação do operador *fish* no código.

`processarSoma`
Percorre a lista de pares de dígitos aplicando a transição monádica.

`stringParaDigitos`
Converte uma `String` em `Maybe [Digito]`, sinalizando caracteres inválidos via `Nothing`.

`somaDois`
Soma duas strings de dígitos usando o autômato monádico; retorna `Nothing` em caso de entrada inválida.

`somaLista`
Generaliza `somaDois` para N strings via `foldl`.

`alinharDigitos`
Preenche as listas de dígitos com `D0` à esquerda.

7 Conclusão

O projeto demonstra, de forma integrada, como uma FSM de Mealy para soma decimal pode ser formalizada em Haskell com:

- **ADTs** que modelam fielmente os conjuntos Q e Σ;
- **Pattern matching exaustivo** que garante completude;
- **State Monad** que encapsula o estado de forma pura;
- **Categoria de Kleisli** que fornece a composição abstrata;
- **Pureza simbólica** que mantém a operação no domínio dos símbolos finitos, sem conversões numéricas.

A fundamentação categorial [2] e o rigor do sistema de tipos de Haskell [1] garantem que o código não apenas funciona, mas é *estruturalmente correto* por construção. A adoção do tipo `Maybe` na fronteira de entrada assegura que todas as funções sejam totais, eliminando exceções em tempo de execução.

Referências

[1] G. Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016.

[2] B. Milewski. *Category Theory for Programmers*. Compilado e editado por I. Tabachnik, 2020.

[3] P. Wadler. Comprehending Monads. *Mathematical Structures in Computer Science*, 2(4):461–493, 1992.

[4] A. G. de Oliveira. *Haskell: Uma Introdução à Programação Funcional*. Casa do Código, 2017.

[5] M. Perone and G. Karachalias. Crème de la Crem: Composable Representable Executable Machines. In *FUNARCH '23*, 2023.

[6] R. L. Bocchino Jr. Monads, Categories, and Computation. 2025.